

# Exercícios: composição

Onze exercícios, seis checkpoints. Cada um diz o que fazer, em qual arquivo, e como saber se deu certo.

---

## **Esta é a lista de exercícios.**

A explicação de cada conceito está no `teoria-composicao.pdf`, nesta mesma pasta. Se travar, o `ajuda-composicao.md` repete os pontos difíceis com palavras mais simples, na pasta Ajuda Extra.

## ANTES DE COMEÇAR

# A sua pasta

Até aqui você tinha três arquivos. Nesta parte entra **um só**, e ele se chama `itens.py`.

```
rpg/
  personagem.py    a classe Personagem
  classes.py       Guerreiro, Mago, Goblin, Orc, Troll, Dragao
  itens.py         NOVO: Item, PocaDeVida, Bomba, Inventario
  main.py          o menu e o combate
```

## QUATRO ARQUIVOS

Não existe `itens2.py`. Não existe `item.py` no singular. Não existe `inventario.py` separado. Se você tem arquivo sobrando, apague antes de começar.

Você precisa do jogo da aula passada rodando, com `_vida`, `curar` e o Dragão. Se o seu quebrou, pegue os três arquivos prontos em `Ajuda Extra/Codigo Base/` e comece de lá.

## Como cada exercício funciona

| PARTE                | O QUE FAZER                                                                    |
|----------------------|--------------------------------------------------------------------------------|
| <b>Arquivo</b>       | Diz onde você escreve, e o desenho mostra a pasta naquele momento.             |
| <b>Faça</b>          | A lista numerada. Uma ação por linha.                                          |
| <b>Pronto quando</b> | Como conferir. Se bater, siga. Se não bater, a caixa diz o erro mais provável. |
| <b>Checkpoint</b>    | Fecha uma etapa. Confira os itens antes de continuar.                          |

**Travou?** Leia a seção com o mesmo assunto no `teoria-composicao.pdf`. A seção 9 de lá lista os erros mais comuns ao compor.

# Sumário

---

|    |                               |    |
|----|-------------------------------|----|
| 01 | O problema: a poção que ataca | 3  |
| 02 | A classe Item                 | 4  |
| 03 | Dois itens, um comando        | 5  |
| 04 | A mochila                     | 7  |
| 05 | Ligar ao personagem           | 8  |
| 06 | Duas ferramentas do Python    | 11 |
| 07 | Fechar o jogo                 | 13 |

---

## CAPÍTULO 1

# O problema: a poção que ataca

---

## Exercício 1: a poção que herda

ARQUIVO: CLASSES.PY, E É TEMPORÁRIO

```
rpg/  
  personagem.py    nao mexa  
  classes.py       <-- voce esta aqui  
  itens.py         ainda nao existe  
  main.py
```

Antes da forma certa, faça a errada. Este exercício existe para você sentir o incômodo.

### Faça:

1. No fim do `classes.py`, crie a classe `Pocao`, herdando de `Personagem`.
2. No `__init__` dela, chame a mãe com: `"Poção de vida", 1, 0, 0, 0`.
3. Crie uma poção e um Goblin.
4. Mande a poção atacar o Goblin.
5. Mande a poção se defender.
6. Pergunte se a poção está viva.

**PRONTO QUANDO**

Tudo funciona. Nenhum erro na tela. A poção ataca, se defende e está viva.

**Esse é o problema.** O Python não te avisa de nada. Uma poção que ataca não faz sentido nenhum, e mesmo assim o programa roda.

Responda antes de seguir: uma poção precisa de `ataque`? Precisa de `defesa`? Precisa de uma mochila dentro dela?

**A PERGUNTA QUE RESOLVE**

Diga em voz alta: "**poção é um personagem?**" Se a frase soar estranha, herança está errada. A frase certa aqui é outra: "**personagem tem poções.**"

Agora apague a classe `Pocao`. Ela não vai ser usada.

## CAPÍTULO 2

# A classe Item

## Exercício 2: o arquivo novo e a classe Item

**ARQUIVO: ITENS.PY, CRIE AGORA**

```
rpg/
  personagem.py    nao mexa
  classes.py
  itens.py         <-- CRIE este arquivo
  main.py
```

**Faça:**

1. Crie o arquivo `itens.py`, na mesma pasta dos outros três.
2. Dentro dele, crie a classe `Item`. Ela **não herda de nada**. A linha é só `class Item:`.
3. Dê a ela um `__init__` que recebe `self` e `nome`, e guarda o nome.
4. Dê a ela um método `usar` que recebe `self`, `dono` e `inimigo`. Por enquanto ele só imprime que o item não faz nada.

**Por que o `usar` recebe os dois?** Porque alguns itens afetam quem usou (a poção) e outros afetam o adversário (a bomba). Se todos recebem os dois, cada item escolhe sozinho quem ele mexe.

#### PRONTO QUANDO

Você cria um `Item("Pedra")`, imprime o nome dele, e depois tenta `item.atacar`. Aí sim dá erro:

```
AttributeError: 'Item' object has no attribute 'atacar'
```

**O erro é o sucesso.** Um item não ataca. Compare com o Exercício 1, onde a poção atacava sem reclamar.

#### CHECKPOINT 1

Existe o arquivo `itens.py`, com a classe `Item` dentro.

`Item` não herda de `Personagem` nem de nada.

A classe `Pocao` do Exercício 1 foi apagada.

A sua pasta tem exatamente quatro arquivos.

## CAPÍTULO 3

# Dois itens, um comando

Agora os itens de verdade. Os dois herdam de `Item`, e desta vez a frase "é um" é verdadeira: poção **é um** item, bomba **é um** item.

## Exercício 3: a poção

### ARQUIVO: ITENS.PY

```
rpg/
  personagem.py    nao mexa
  classes.py
  itens.py         <-- voce esta aqui
  main.py
```

### Faça:

1. Crie a classe `PocaoDeVida`, herdando de `Item`.
2. O `__init__` dela não recebe nada além do `self`. Chame a mãe com o nome `"Poção de vida"`.
3. Depois disso, guarde `self.cura` valendo `25`.
4. Sobrescreva o `usar`. Ele deve mandar o **dono** se curar, usando o método `curar` que você escreveu na Parte 2.
5. Imprima o que aconteceu.

### NÃO FAÇA A CONTA AQUI

O item não mexe em `dono._vida`. Ele chama `dono.curar(...)` e deixa o personagem aplicar as regras dele.

### PRONTO QUANDO

Um personagem com 50 de vida usa a poção e fica com 75.

## Exercício 4: a bomba

### ARQUIVO: ITENS.PY

### Faça:

1. Crie a classe `Bomba`, herdando de `Item`, com o nome `"Bomba"` e `self.dano` valendo `30`.
2. Sobrescreva o `usar`. Ele deve mandar o **inimigo** receber dano, usando `receber_dano`.
3. Imprima o que aconteceu.

Repare: o `usar` da bomba tem **exatamente os mesmos parâmetros** do `usar` da poção. Só o corpo muda.

**PRONTO QUANDO**

Você põe uma poção e uma bomba numa lista, percorre a lista chamando `item.usar(heroi, bicho)` nas duas, e o herói sobe de 50 para 75 enquanto o Goblin cai de 40 para 12.

**Olhe o seu laço.** Ele não pergunta que item é. Não tem `if`. É o mesmo polimorfismo da Parte 1, agora em objetos que não são personagens.

**Se o herói não subiu para 75:** a sua poção está mexendo na vida direto em vez de chamar `curar`.

**CHECKPOINT 2**

`PocaoDeVida` e `Bomba` herdam de `Item`.

Os dois `usar` recebem `self`, `dono` e `inimigo`.

A poção mexe só no dono. A bomba mexe só no inimigo.

Nenhum dos dois toca em `_vida` diretamente.

## CAPÍTULO 4

# A mochila

Você poderia guardar os itens numa lista solta dentro do personagem. Vai funcionar. Mas aí a regra de "qual índice é o item 1" e "o que acontece se o número não existe" fica espalhada por todo lugar que abre a mochila.

Uma classe resolve isso: a lista ganha dono.

## Exercício 5: a classe Inventario

**ARQUIVO: ITENS.PY**

```
rpg/
  personagem.py    nao mexa
  classes.py
  itens.py         <-- voce esta aqui
  main.py
```

**Faça:**

1. No mesmo `itens.py`, crie a classe `Inventario`. Ela **não herda de nada**.
2. No `__init__`, crie `self._itens` como uma lista vazia. Com underscore: a lista é assunto interno.
3. Crie `adicionar(self, item)`, que põe o item na lista.
4. Crie `quantidade(self)`, que devolve quantos itens tem.
5. Crie `esta_vazio(self)`, que devolve verdadeiro ou falso.
6. Crie `listar(self)`, que imprime cada item numerado a partir de **1**, não de zero.
7. Crie `tirar(self, numero)`. Ele converte o número para índice, devolve `None` se o número não existir, e senão **remove e devolve** o item.

**Por que `tirar` devolve `None` em vez de quebrar?** Porque quem chama é o menu, e o jogador vai digitar 9 numa mochila de 2 itens.

**PRONTO QUANDO**

Você cria uma mochila, adiciona dois itens, lista, e chama `tirar(99)`. Tem que sair `None`, e a quantidade tem que continuar 2.

A listagem ainda vai sair feia, mais ou menos assim:

```
1 - <itens.PocaoDeVida object at 0x...>
```

Isso é o Python dizendo "não sei como transformar isto em texto". O Exercício 9 conserta. Por enquanto ignore.

**CHECKPOINT 3**

Existe a classe `Inventario`, e ela não herda de nada.

A lista se chama `_itens`, com underscore.

`tirar` remove o item e devolve ele.

`tirar` com número inválido devolve `None` e não quebra.



# Ligar ao personagem

## Exercício 6: o personagem ganha a mochila

ARQUIVO: PERSONAGEM.PY, AGORA VOCÊ MEXE NELE

```
rpg/
  personagem.py    <-- voce esta aqui
  classes.py
  itens.py
  main.py
```

### Faça:

1. No topo do `personagem.py`, importe `Inventario` e `PocaoDeVida` do arquivo `itens`.
2. No `__init__`, **apague** a linha `self.pocoes = pocoes`.
3. No lugar dela, crie `self.inventario` valendo um `Inventario()` novo.
4. Logo abaixo, use um `for` que repete `pocoes` vezes e adiciona uma `PocaoDeVida()` na mochila a cada volta.
5. No `mostrar_status`, troque a linha das poções por uma que mostre `self.inventario.quantidade()`.

### REPRE NO QUE VOCÊ NÃO PRECISOU FAZER

O parâmetro `pocoes` continua existindo. Por isso **nenhuma** classe filha mudou: Guerreiro, Mago, Goblin, Orc, Troll e Dragão continuam iguais. Você trocou o interior do `Personagem` e as seis filhas não ficaram sabendo.

### PRONTO QUANDO

Um Guerreiro nasce com 2 itens, um Mago com 4, e um Goblin com 0. São exatamente os números que já estavam no `super().__init__(...)` de cada classe.

**Se der `TypeError` falando de argumentos:** você tirou o parâmetro `pocoes` do `__init__`. Ele tem que continuar lá. Quem mudou foi o que você faz com ele.

**Se der `AttributeError: 'NoneType'`:** você escreveu `Inventario` sem os parênteses. Sem eles você guardou a classe, não uma mochila.

## Exercício 7: o menu da mochila

### ARQUIVO: MAIN.PY

```
rpg/
  personagem.py
  classes.py
  itens.py
  main.py          <-- voce esta aqui
```

### Faça:

1. No `turno_do_jogador`, troque o texto da opção 3 para `"3 - Abrir a mochila"`.
2. Faça a opção 3 chamar uma função nova, `abrir_mochila(jogador, inimigo)`.
3. Escreva essa função. Se a mochila estiver vazia, avise e saia.
4. Senão, liste os itens e peça um número, avisando que `0` volta.
5. Tire o item escolhido da mochila. Se vier `None`, avise que o item não existe e saia.
6. Se vier um item, mande ele ser usado, passando o jogador e o inimigo.

### PRONTO QUANDO

Você abre a mochila no meio do combate, escolhe uma poção, e recupera vida. Escolher um número que não existe não quebra o jogo.

**Se o item sumir sem efeito:** o `tirar` já removeu o item, mas você esqueceu de chamar o `usar` depois. Tirar e usar são duas ações.

## Exercício 8: apagar o `usar_pocao`

### ARQUIVO: PERSONAGEM.PY

O `usar_pocao` não tem mais função. A mochila faz o trabalho dele, e faz melhor: agora serve para qualquer item, não só poção.

### Faça:

1. Apague o método `usar_pocao` inteiro do `personagem.py`.
2. Rode o jogo e confirme que nada quebrou.

## CHECKPOINT 4

O `Personagem` tem um `self.inventario`.

Nenhuma classe do `classes.py` precisou mudar.

A opção 3 do menu abre a mochila e usa um item.

O método `usar_pocao` não existe mais.

O `main.py` nunca escreve `_itens`.

## CAPÍTULO 6

# Duas ferramentas do Python

## Exercício 9: como o objeto vira texto

ARQUIVO: ITENS.PY, DEPOIS PERSONAGEM.PY

```
rpg/
  personagem.py    <-- depois aqui
  classes.py
  itens.py         <-- comece aqui
  main.py
```

Lembra do `<itens.PocaoDeVida object at 0x...>` do Exercício 5? Existe um método especial para consertar isso. Ele se chama `__str__`, com dois underscores de cada lado, igual ao `__init__`. Você nunca chama ele na mão: o `print` chama por você.

### Faça:

1. Na classe `Item`, crie o método `__str__`, que recebe só `self` e **devolve** o nome do item. Use `return`, não `print`.
2. Liste a mochila de novo e veja ficar legível.
3. Na classe `Personagem`, crie um `__str__` que devolve o nome com a vida atual e a máxima, no formato `Thoric (120/120)`.

**Por que `return` e não `print`?** Porque o `__str__` responde "qual é o seu texto", e quem decide o que fazer com esse texto é quem perguntou. Se você usar `print`, o método devolve `None` e o Python reclama.

**PRONTO QUANDO**

```
print(pocao)      # Poção de vida
print(heroi)      # Thoric (120/120)
```

**Se ainda aparecer o endereço de memória:** o nome do método está errado, ou ele ficou fora da classe. Confira a indentação e os dois underscores de cada lado.

## Exercício 10: a porta de leitura

**ARQUIVO: PERSONAGEM.PY**

Na Parte 2 você fechou a porta trocando `self.vida` por `self._vida`. Isso resolveu o `inimigo.vida = -999`, mas cobrou um preço: hoje nem **ler** a vida funciona de fora.

Ler nunca foi o problema. O problema era escrever.

**Faça:**

1. Na classe `Personagem`, escreva a linha `@property` sozinha.
2. Logo abaixo dela, crie um método chamado `vida`, que recebe só `self` e devolve `self._vida`.
3. Faça o mesmo para `vida_maxima`.

**PRONTO QUANDO**

`print(heroi.vida)` mostra `120`, e `heroi.vida = 999` dá este erro:

```
AttributeError: property 'vida' of 'Guerreiro' object has no setter
```

Ler liberado, escrever bloqueado. É exatamente isso que você queria.

Repare que quem usa não mudou nada: continua escrevendo `heroi.vida`, sem parênteses, como se fosse atributo comum.

**CHECKPOINT 5**

`Item` tem `__str__`, e a mochila lista nomes legíveis.

`Personagem` tem `__str__`.

`personagem.vida` pode ser lido de fora.

`personagem.vida = 999` continua proibido.

# Fechar o jogo

## Exercício 11: o inimigo deixa um item

### ARQUIVO: MAIN.PY

```
rpg/  
  personagem.py  
  classes.py  
  itens.py  
  main.py          <-- voce esta aqui
```

Falta um jeito de ganhar itens durante o jogo. Senão você começa com duas poções e acabou.

### Faça:

1. Importe `Bomba` e `PocaoDeVida` do `itens`.
2. No laço dos inimigos, logo depois do descanso de 50 de vida, escolha um prêmio: uma `Bomba()` se o inimigo derrotado foi o Troll, e uma `PocaoDeVida()` nos outros casos.
3. Adicione o prêmio à mochila do jogador.
4. Avise na tela o que caiu. Como o item tem `__str__`, dá para imprimir o objeto direto.

### PRONTO QUANDO

Você derrota o Goblin e vê na tela que ele deixou uma poção, e a sua mochila passa a ter um item a mais.

### CHECKPOINT 6, O ÚLTIMO

O jogo roda do início ao fim: escolhe personagem, enfrenta os quatro inimigos, e termina com vitória ou derrota.

Dá para abrir a mochila no meio do combate e usar um item.

Cada inimigo derrotado deixa um item para trás.

Nenhum arquivo fora do `personagem.py` escreve `_vida`.

Nenhum arquivo fora do `itens.py` escreve `_itens`.

**TESTE FINAL**

Abra `main.py`, `classes.py` e `itens.py` e procure por `_vida` com Ctrl+F. Depois procure por `_itens` no `main.py` e no `personagem.py`.

**Não pode aparecer nenhuma vez.** Se aparecer, tem código de fora mexendo no que não é dele. Troque por uma chamada de método.

## O que você construiu

| CONCEITO        | ONDE ESTÁ NO SEU JOGO                                                                    |
|-----------------|------------------------------------------------------------------------------------------|
| Classe e objeto | <code>Personagem</code> , <code>Item</code> , <code>Inventario</code> .                  |
| Herança         | Guerreiro, Mago, Dragão, e também <code>PocaoDeVida</code> e <code>Bomba</code> .        |
| Polimorfismo    | <code>inimigo.atacar()</code> no combate, <code>item.usar()</code> na mochila.           |
| Encapsulamento  | <code>_vida</code> e <code>curar</code> ; <code>_itens</code> e <code>adicionar</code> . |
| Composição      | <code>Personagem</code> tem um <code>Inventario</code> , que tem <code>Item</code> .     |

São os cinco conceitos da matéria, todos rodando ao mesmo tempo, num programa que você joga do começo ao fim.

**Terminou tudo?** As perguntas finais do `teoria-composicao.pdf` fecham o assunto. Responda elas com o seu código aberto do lado.